Unit – 4

Software Quality and Metrics

Quality Attributes • Quality Models • Measurement Basics • Metric Analysis • Size/Structure Metrics • Quality Metrics • OO Metrics



MCA – 212 • Object-Oriented Software Engineering

Dr Madhavi Dave

Learning Objectives

What students should be able to do after this module

01 • Quality Attributes

- Name and classify the internal and external attributes that define software quality.

02 • Quality Models

- Compare McCall's, Boehm's and ISO 9126 quality models.

03 • Measurement Basics

- Distinguish measures, metrics and indicators, and apply measurement scales correctly.

04 • Metric Data Analysis

- Summarize and visualize metric data; spot outliers and analysis pitfalls.

05 • Size, Structure & Quality Metrics

- Compute cyclomatic complexity, coupling/cohesion, defect density and DRE.

06 • OO Metrics

- Compute and interpret the CK metrics suite for a class design.

- The Library Management System from Unit 3 returns as our running case study for size, structure and OO metrics.

SECTION 1

Software Quality Attributes

Naming the properties that separate good software from bad

What Is Software Quality?

Conformance to requirements — and to the needs no one wrote down

- Software quality is conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics expected of professionally developed software.
- "Implicit characteristics" is the tricky part: users expect maintainability, usability and reliability even when nobody wrote a requirement for them.
- Quality cannot be inspected in at the end — it has to be designed in and measured throughout the life cycle.

Reference

Pressman, Software Engineering: A Practitioner's Approach — chapters on Software Quality and Metrics for Process, Project and Product form the basis of this unit.

Quality Attributes — McCall's 11 Factors

Grouped into three concerns: running it, changing it, and moving it

Product Operation

- Correctness
- Reliability
- Efficiency
- Integrity
- Usability

Product Revision

- Maintainability
- Flexibility
- Testability

Product Transition

- Portability
- Reusability
- Interoperability

How to read this

"Operation" factors matter while the system runs; "Revision" factors matter while the system is being changed; "Transition" factors matter when it moves to a new environment.

Internal vs External Quality Attributes

What developers see vs what users experience

Aspect	Internal Attributes	External Attributes
Visible to	Developers, reviewers, static analysis tools	End users, customers, operators
Measured from	Source code, design documents	System behavior in use
Examples	Coupling, cohesion, cyclomatic complexity	Reliability, usability, response time
Relationship	Good internal attributes	→ tend to produce good external ones

Why the distinction matters

Internal attributes are what a team can directly control during development; external attributes are what actually gets judged in production. Metrics let us predict the external from the internal.

SECTION 2

Software Quality Models

Structured frameworks for turning "quality" into something measurable

McCall's Quality Model

Factors (what) → Criteria (how) → Metrics (measure)

Factors
(what the user wants)

→

Criteria
(how it's achieved)

→

Metrics
(how it's measured)

Product Operation

- Correctness, Reliability, Efficiency, Integrity, Usability

Product Revision

- Maintainability, Flexibility, Testability

Product Transition

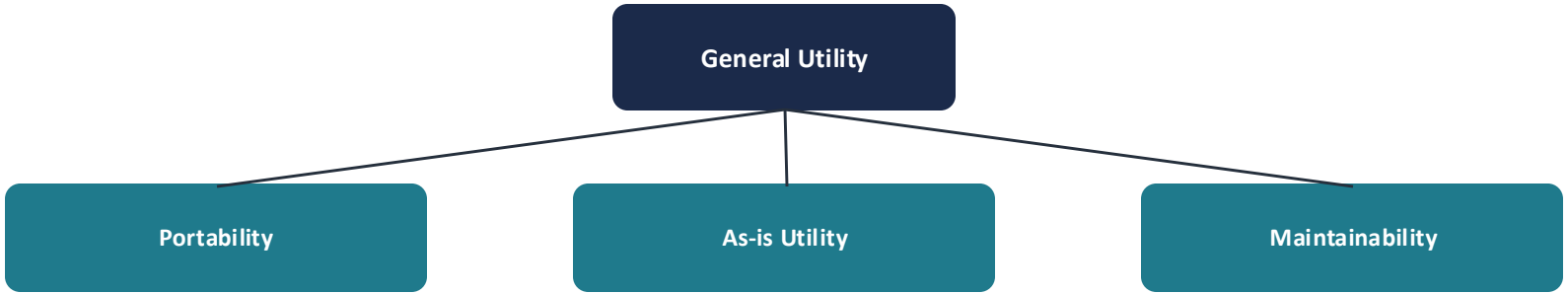
- Portability, Reusability, Interoperability

Example chain

Factor: Maintainability → Criterion: Modularity (a design attribute) → Metric: average module coupling — a low coupling count is evidence the maintainability factor is being achieved.

Boehm's Quality Model

General Utility, decomposed into usable, as-is, and maintainable



Branch	Decomposes into
Portability	Device-independence, Self-containedness
As-is Utility	Reliability, Efficiency, Human Engineering
Maintainability	Testability, Understandability, Flexibility

ISO/IEC 9126 Quality Model

Six characteristics — the basis for today's ISO/IEC 25010 (SQuaRE)

Characteristic	Sub-characteristics (examples)
Functionality	Suitability, Accuracy, Interoperability, Security
Reliability	Maturity, Fault tolerance, Recoverability
Usability	Understandability, Learnability, Operability
Efficiency	Time behavior, Resource utilization
Maintainability	Analyzability, Changeability, Stability, Testability
Portability	Adaptability, Installability, Replaceability

Successor

ISO/IEC 25010 (2011) expanded this to 8 characteristics — adding Compatibility and Security as first-class citizens rather than sub-characteristics.

Comparing the Three Models

Different eras, same underlying goal

Model	Origin	Structure	Notable For
McCall's	1977	11 factors / 23 criteria / metrics	First influential factor model
Boehm's	1978	Hierarchical tree of utility	Adds "as-is utility" concept
ISO 9126 → 25010	1991 → 2011	6 → 8 characteristics	International standard, widely adopted

In practice

Modern teams rarely quote these models by name, but every quality checklist in use today — accessibility audits, non-functional requirement templates, code review standards — traces back to one of them.

SECTION 3

Measurement Basics

The vocabulary and scales every metric is built from

What Is Measurement?

Direct measures vs. indirect measures

Direct Measures

Immediately observable, without reference to any other measure.

Examples: lines of code, execution speed, number of defects reported, effort in person-hours.

Simple to collect, but rarely tells the whole story on its own.

Indirect Measures

Derived by combining direct measures through a formula or model.

Examples: quality, complexity, efficiency, reliability, maintainability.

Requires a validated model — a bad formula produces a misleading number just as confidently as a good one.

Measurement Scales

Not all numbers mean the same kind of thing

Scale	Property	Example
Nominal	Categories only, no order	Defect type: syntax / logic / interface
Ordinal	Ordered categories, gaps not meaningful	Severity: low / medium / high / critical
Interval	Ordered, equal gaps, no true zero	Calendar date, a 1–10 subjective rating
Ratio	Ordered, equal gaps, true zero	LOC, defect count, effort in hours
Absolute	A direct count; only the identity transform applies	Number of test cases executed

Why it matters

Averaging ordinal severity ratings ("average severity = 2.6") is a classic measurement error — ordinal scales don't support arithmetic mean, only mode or median.

Measures, Metrics & Indicators

Raw numbers become insight only in context



Measure

- "1,500 lines of code were written this sprint."

Metric

- "Defects per KLOC = 2.3 this release."

Indicator

- "Defect rate is up 20% vs. last release → investigate before shipping."

SECTION 4

Analysis of Metric Data

Turning collected numbers into a defensible conclusion

Descriptive Statistics for Metric Data

The first pass over any metric dataset

Statistic	Tells You	Watch Out For
Mean	The arithmetic average	Distorted badly by outliers
Median	The middle value	Robust to outliers — often more honest
Mode	The most frequent value	Only useful with enough repeated values
Std. Deviation	How spread out the data is	Assumes a roughly symmetric distribution

Worked example — defects per module: 5, 8, 12, 5, 20

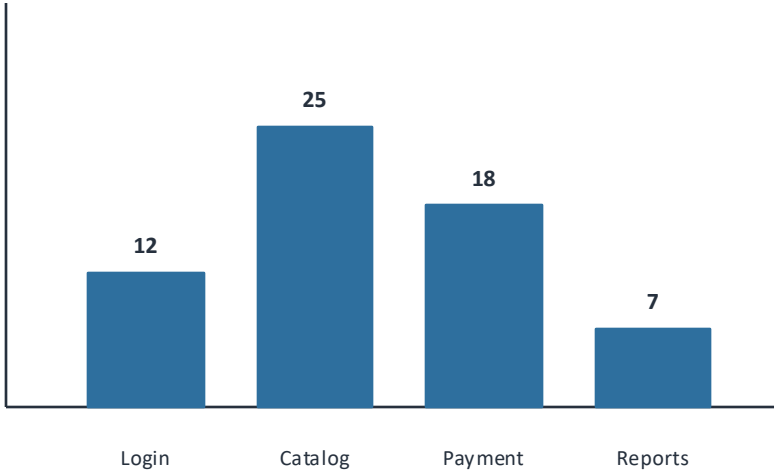
Mean = 10 • Median = 8 • Mode = 5 • Range = 15 • Sample std. dev. ≈ 6.28

The mean (10) is pulled upward by the outlier module with 20 defects — the median (8) is the more representative "typical" value here.

Visualizing Metric Data

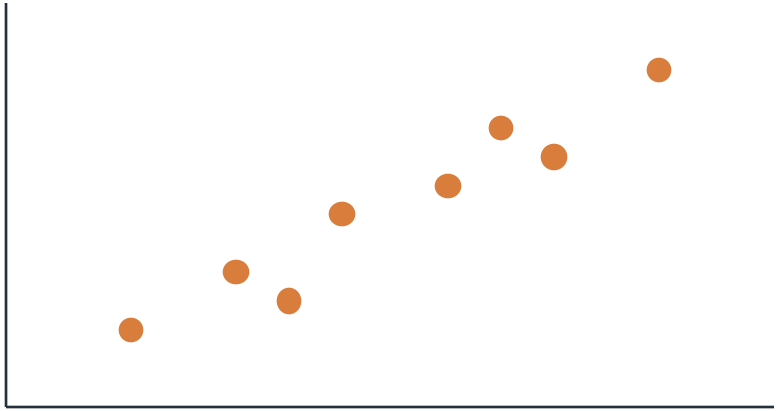
A histogram and a scatter plot answer different questions

Histogram — Defects by Module



Shows where defects concentrate — Catalog clearly needs the most review attention.

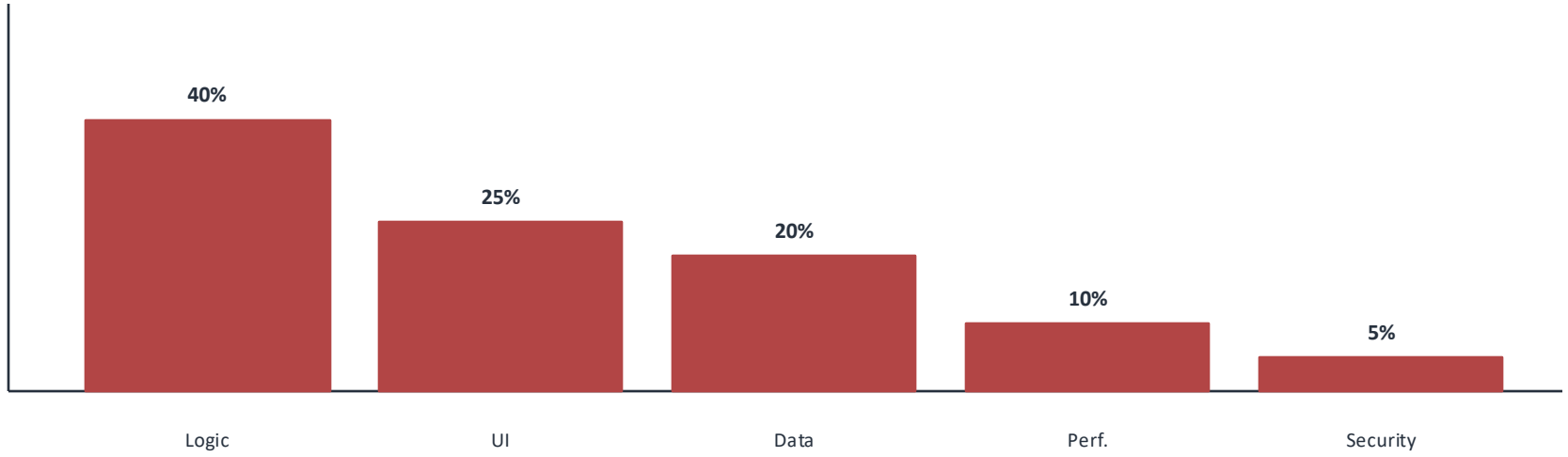
Scatter Plot — Module Size vs. Defects



Larger modules (in KLOC) tend to have more defects — a common, testable hypothesis.

Case Study — Pareto Analysis of Defect Categories

Focus review effort where it pays off most



The 80/20 rule in action

Logic and UI defects together account for 65% of all defects found. A Pareto analysis like this tells a team exactly where to focus code review and testing effort first.

Outliers & Common Pitfalls

Metric analysis is easy to get subtly wrong

Outlier distortion

- One unusual module can dominate the mean — always check the median too.

Correlation ≠ causation

- Larger modules having more defects doesn't prove size causes defects.

Survivorship bias

- Only reported defects get counted — silently-tolerated bugs are invisible.

Goodhart's Law

- "When a measure becomes a target, it ceases to be a good measure."

Small sample sizes

- Drawing conclusions from 3–4 data points is rarely statistically sound.

Cherry-picked windows

- Choosing a favorable date range can make any trend look good.

SECTION 5

Metrics for Measuring Size and Structure

From how big the code is, to how tangled it is

Size Metrics — A Quick Recap

Two ways to answer "how big is this software?"

Metric	Strength	Weakness
Lines of Code (LOC)	Simple, easy to automate	Language-dependent; needs code to exist
Function Points (FP)	Language-independent; usable early	Requires trained counters; more effort

Already covered

LOC, Function Points, Use Case Points, Class Points and OO Function Points were covered in detail as estimation techniques in the earlier unit — the same size numbers now double as quality-metric denominators (e.g. defects per KLOC).

Complexity Metric — McCabe's Cyclomatic Complexity

A structural metric computed straight from the flow graph

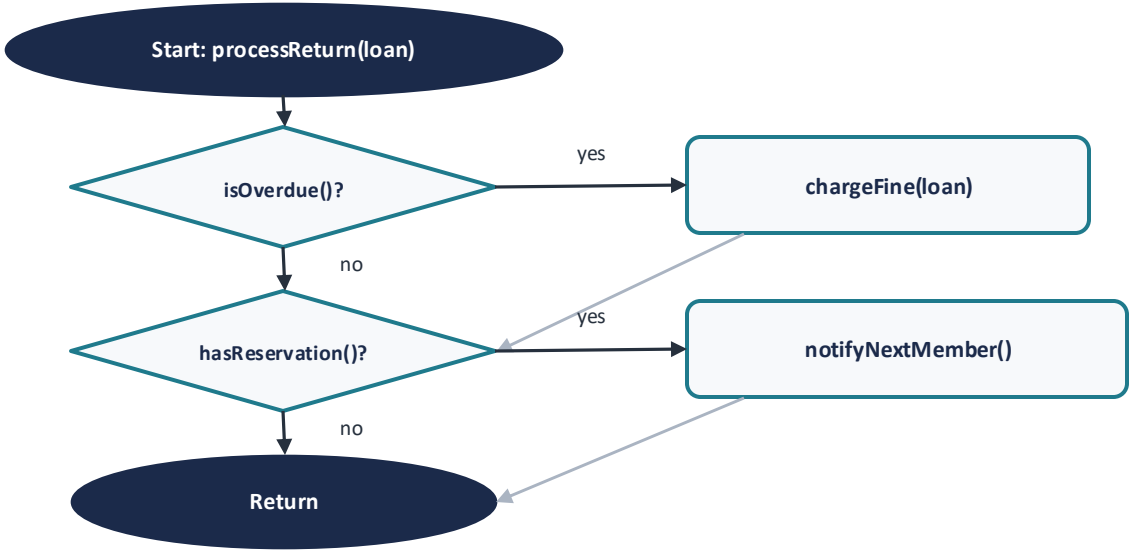
V(G) = E – N + 2 (E = edges, N = nodes)

V(G) = D + 1 (D = number of decision points)

- Both formulas always agree — use whichever is easier to count from your diagram or code.
- V(G) also equals the number of independent paths through the code, and therefore the minimum number of test cases needed for full branch coverage.
- Rule of thumb: V(G) ≤ 10 is manageable; V(G) > 20 signals a method that urgently needs to be broken up.

Case Study — Flow Graph for processReturn(loan)

Two decision points → V(G) = 3



N = 6 nodes, E = 7 edges → V(G) = 7 − 6 + 2 = 3. Two decision nodes → V(G) = 2 + 1 = 3. ✓

Structure Metrics — Coupling & Cohesion

Two classic module-design measures, still relevant in OO design

Coupling (aim: low)

Content — worst: one module reaches into another's internals

Common — modules share global data

Control — one module dictates another's logic via a flag

Stamp — a whole data structure is passed, not just what's needed

Data — best: only simple data parameters are passed

Cohesion (aim: high)

Coincidental — worst: unrelated tasks grouped by accident

Logical — related only by category, not behavior

Temporal — grouped only because they run at the same time

Procedural / Communicational — share control flow or data

Functional — best: every part serves one single purpose

SECTION 6

Measuring Software Quality

Turning defect data into two of the most-used quality metrics

Defect-Based Quality Metrics

Defect Density and Defect Removal Efficiency

Defect Density = Total Known Defects ÷ Size (KLOC or FP)

DRE = E ÷ (E + D) (E = errors found pre-release, D = defects found post-release)

- Defect Density lets teams compare quality across modules or releases of different sizes on a level footing.
- DRE (Defect Removal Efficiency) measures how good your testing and review process is at catching problems before customers do — closer to 100% is better.
- Track both over time, not just once: a single snapshot rarely tells you whether quality is improving or slipping.

Case Study — Defect Density & DRE for the Library System

Turning raw testing data into two quality metrics

Given

Module size = 12 KLOC

Errors found during in-house testing (E) = 45

Defects reported by users in first 3 months (D) = 5

Total known defects = 45 + 5 = 50

Computed

Defect Density = 50 ÷ 12 ≈ 4.17 defects / KLOC

DRE = 45 ÷ (45 + 5) = 45 ÷ 50 = 0.90 = 90%

90% of defects were caught before release — a strong, though not perfect, result.

Reading the result

A DRE of 90% is generally considered good industry practice; teams aiming for safety-critical quality typically target 95%+ by adding more review and test stages before release.

SECTION 7

Object-Oriented Metrics

The CK metrics suite — measuring classes, not just lines

Why OO Needs Its Own Metrics

Inheritance, coupling and encapsulation don't show up in LOC counts

- Traditional size and complexity metrics (LOC, cyclomatic complexity) were designed for procedural code — they say nothing about a class hierarchy or message-passing structure.
- OO-specific quality depends on how responsibility is distributed across classes: how deep the inheritance goes, how many classes each one depends on, how cohesive each class's methods are.
- Chidamber & Kemerer (1994) proposed six metrics — the CK suite — specifically to fill this gap, and it remains the most widely cited OO metrics suite today.

The CK Metrics Suite

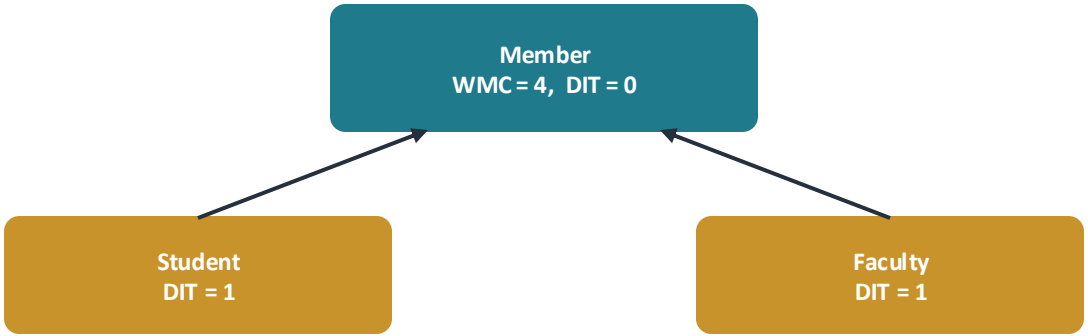
Six metrics, one for each dimension of class design

Metric	Measures	Goal
WMC	Weighted Methods per Class — sum of method complexities	Lower is simpler
DIT	Depth of Inheritance Tree — distance from the root class	Moderate is best
NOC	Number of Children — immediate subclasses	Depends on design
CBO	Coupling Between Objects — classes it depends on	Lower is better
RFC	Response For a Class — methods reachable from one call	Lower is better
LCOM	Lack of Cohesion in Methods	Lower is better

WMC & DIT

Weighted Methods per Class, and Depth of Inheritance Tree

WMC = Σ complexity(method) for every method defined in the class

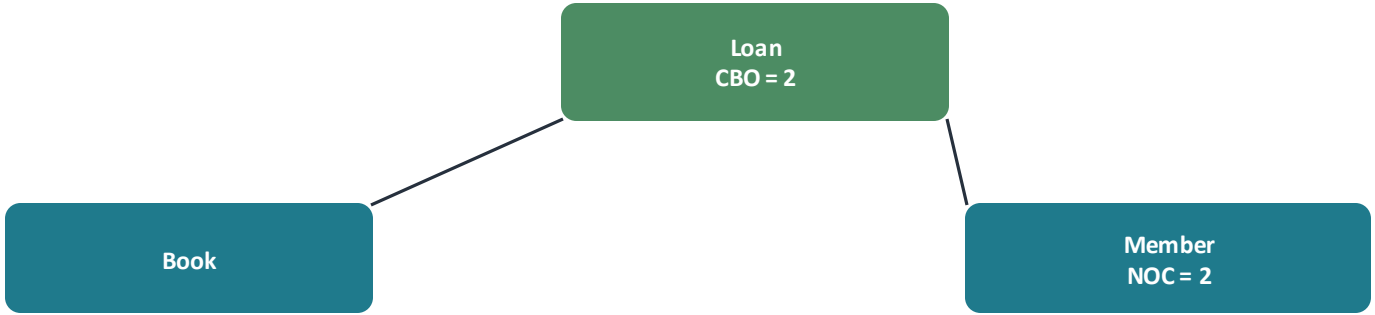


DIT is the length of the longest path to the root: Member sits at the root (DIT=0); Student and Faculty are one level below it (DIT=1).

NOC & CBO

Number of Children, and Coupling Between Objects

CBO = count of distinct classes a class is coupled to (uses or is used by)



Loan is coupled to 2 classes (Book, Member) → CBO = 2. Member has 2 immediate subclasses, Student and Faculty (see previous slide) → NOC = 2.

RFC & LCOM

Response For a Class, and Lack of Cohesion in Methods

RFC = |M ∪ R| M = class's own methods, R = methods they call on other objects

LCOM = pairs of methods NOT sharing an attribute − pairs that DO (if positive)

Worked example — Loan class

Loan's own methods (M): calculateFine(), extend(), close() → |M| = 3.

Methods it calls elsewhere (R): book.checkAvailability(), member.notify() → |R| = 2.

RFC = |M ∪ R| = 5 — a small, easily-understood response set.

Low LCOM (close to 0) means the class's methods work together on the same data — a genuinely cohesive class.

Case Study — CK Metrics for the Library System

Same six classes from the Unit 3 class diagram

Class	WMC	DIT	NOC	CBO	RFC	LCOM
Library	6	0	0	2	9	1
Book	5	0	0	2	7	0
Member	4	0	2	3	8	1
Student	2	1	0	1	5	0
Faculty	2	1	0	1	5	0
Loan	5	0	0	2	5	0

Reading the table
No class stands out as risky: WMC and CBO stay low across the board, and only Member (the inheritance root) carries a non-zero NOC — exactly what a clean, shallow design should look like.

Interpreting OO Metrics

Commonly cited rule-of-thumb thresholds

Metric	Watch-out threshold	What high values usually mean
WMC	**> 20**	Class is doing too much — split responsibilities
DIT	**> 5**	Hierarchy too deep to reason about safely
NOC	**> 10**	Base class may be too generic / overused
CBO	**> 14**	Class is entangled with too many others
RFC	**> 50**	Interface is too large to test or understand
LCOM	**High (near 1)**	Class likely doing unrelated jobs — split it

Use thresholds as smoke alarms, not laws

A class exceeding a threshold isn't automatically wrong — it's a prompt to ask why, and to look closer during code review.

How It All Connects

From an abstract idea of "quality" to a number on a dashboard



- A quality attribute (e.g. maintainability) only becomes actionable once a quality model breaks it into criteria and metrics.
- Every metric must be collected on the right measurement scale, or its statistics are meaningless.
- Raw metric data becomes a decision-ready indicator only after proper analysis — descriptive stats, visualization, and outlier checks.
- For OO systems specifically, the CK suite supplies the structural metrics that traditional LOC/complexity measures miss entirely.

Key Takeaways

Quality is designed in, and metrics are how you know it happened

- Quality models (McCall's, Boehm's, ISO 9126/25010) exist to turn a vague word like "quality" into a checklist of measurable criteria.
- Know your measurement scale before you compute a statistic — averaging ordinal data is a common and avoidable mistake.
- Structural metrics (cyclomatic complexity, coupling, cohesion, and the CK suite for OO) predict maintenance pain before it happens.
- Defect Density and DRE turn testing records into two of the most direct, defensible quality metrics a team can report.